Міністерство освіти та науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

ЗВІТ

з дисципліни: «Програмування веб-застосувань»

Практична робота №11

Локальні сховища даних: localStorage/IndexedDB

Виконав:
Студент II курсу
групи ІП-56
Гончар І.В.

Перевірив:
Ковтунець О.В.

2026

1. **Мета роботи:**

Навчитися зберігати дані застосунку між перезавантаженнями сторінки: спочатку найпростішим способом - через localStorage (JSON.stringify/parse), а потім перейти на просту схему IndexedDB з одним об'єктним сховищем і базовими CRUD-операціями (створення, читання, оновлення, видалення). Окрема увага - різниці між синхронним доступом (localStorage блокує потік виконання) і асинхронним (IndexedDB працює через транзакції та колбеки/Promise), а також одноразовій міграції наявних даних із localStorage в IndexedDB.

2. **Хід виконання роботи**

Крок 1-4. Модель даних та реалізація збереження в localStorage:

Для предметної області "Каталог настільних ігор" (Варіант 4) до масиву об'єктів було додано поле owned (статус «Зіграно» / «Не зіграно») та унікальний ідентифікатор id. Реалізовано функції saveToLocalStorage та loadFromLocalStorage з використанням JSON.stringify та JSON.parse (загорнутого у блок try/catch для обробки пошкоджених даних). У компоненті Vue налаштовано спостерігач (watch) з параметром deep: true, який автоматично зберігає масив ігор у localStorage після кожного кліку по кнопці зміни статусу.

Дані успішно зберігаються між перезавантаженнями сторінки, що підтверджується перевіркою у панелі DevTools (рис. 1).

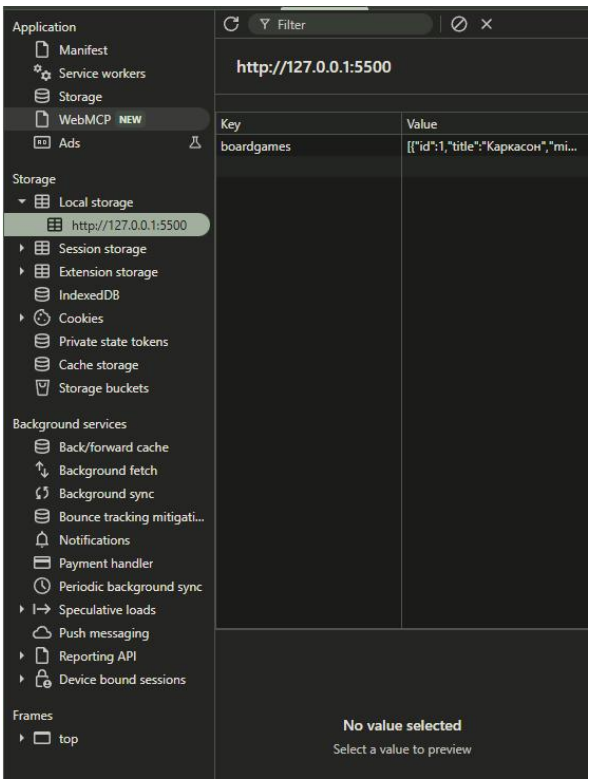


Рис. 1 — Перевірка збереження даних у localStorage

Крок 5-6. Перевірка збереження стану інтерфейсу: Для підтвердження того, що дані успішно зберігаються не лише в панелі DevTools, а й коректно

відновлюються для користувача, було проведено тестування інтерфейсу застосунку. На Рис. 2 зображено початковий стан карток (статус «Не зіграно»). Після зміни статусу однієї з ігор на «Зіграно» сторінку було перезавантажено (F5). Як видно на Рис. 3, стан кнопки успішно зберігся, що підтверджує коректну роботу інтеграції Vue watch (з deep: true) та синхронного збереження у localStorage.

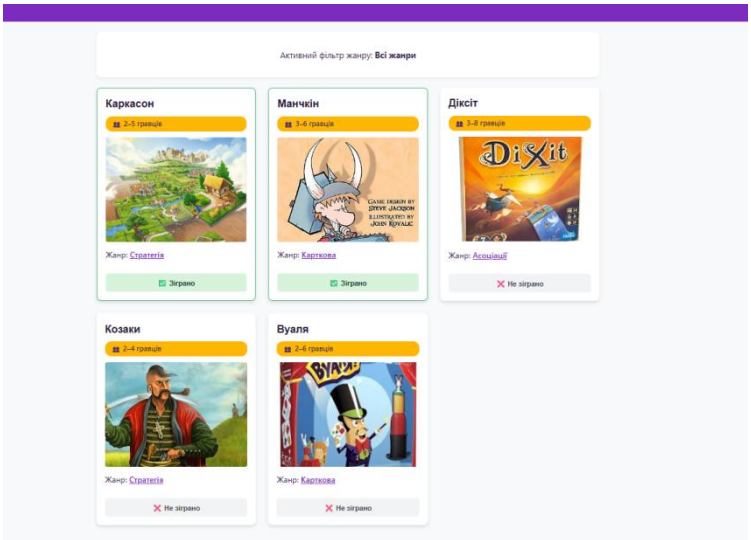


Рис. 2 — Інтерфейс застосунку до зміни статусу

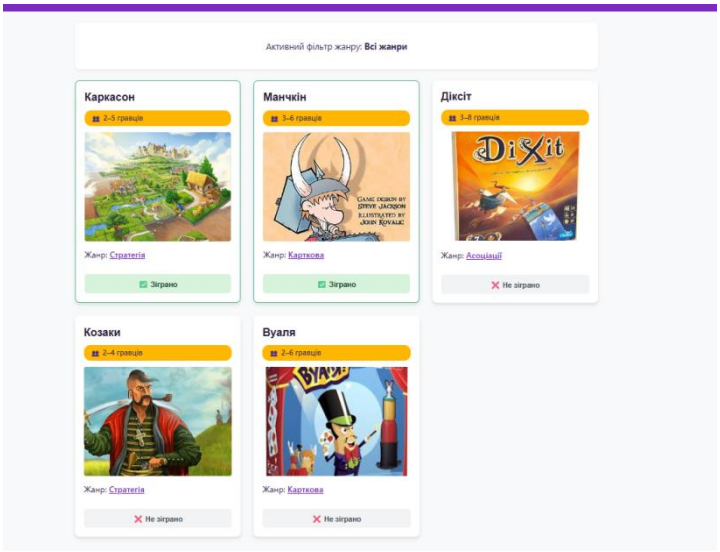


Рис. 3 — Інтерфейс застосунку після перезавантаження (дані збережено)

Крок 7. Перевірка бази даних IndexedDB: Відповідно до завдання, було реалізовано міграцію даних у повноцінну базу IndexedDB (BoardGamesDB) зі створенням object store games та налаштуванням CRUD-транзакцій. Успішне створення бази та збереження даних підтверджено через панель розробника (Рис. 4).

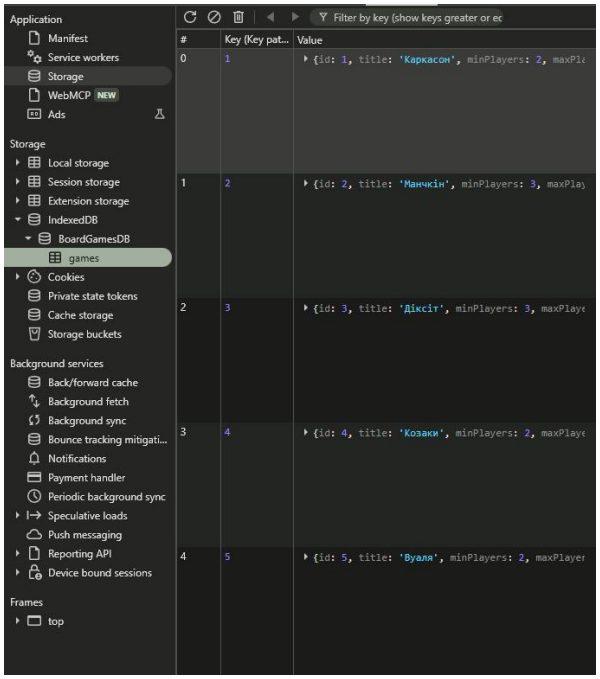


Рис. 4 — Перевірка структури та даних у сховищі IndexedDB

Крок 8. Підтвердження міграції даних та обробка помилок: Реалізовано механізм одноразової міграції даних із `localStorage` до `IndexedDB`. Після успішного перенесення масиву ігор у базу даних, у `localStorage` встановлюється спеціальний прапорець `migratedToIDB: true`, а в консоль виводиться відповідне повідомлення про успіх міграції. Також додано візуальне сповіщення користувача (`alert`) у блоці `try/catch` на випадок помилки ініціалізації БД. Це підтверджує правильну обробку помилок та успішне перенесення даних (Рис. 5).

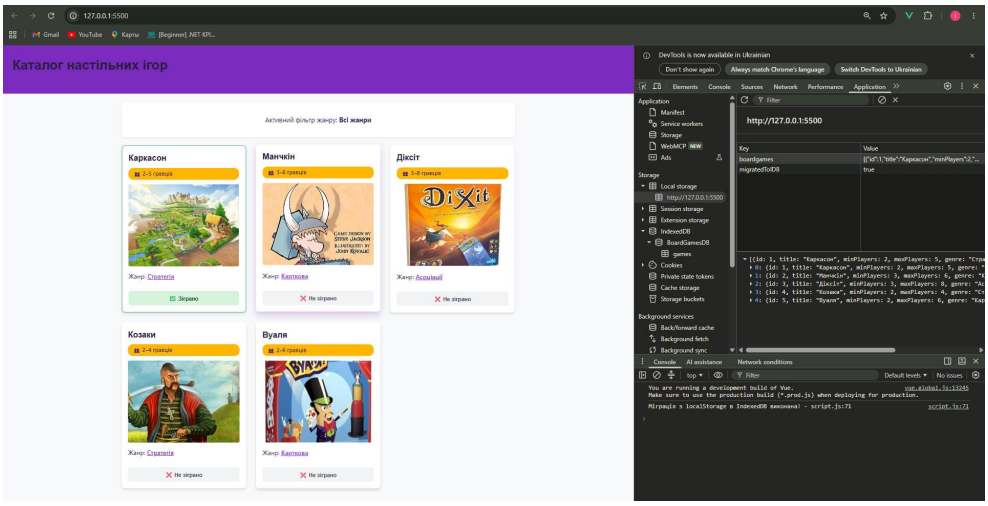


Рис. 5 — Підтвердження міграції даних (повідомлення в консолі та прапорець у localStorage)

Крок 9-10. Версіонування та збереження: Усі оновлення коду, що стосуються впровадження локальних баз даних (localStorage та IndexedDB), логіки міграції збережених записів та обробки помилок, було успішно зафіксовано системою контролю версій Git. Зміни супроводжено описовим повідомленням та відправлено до віддаленого репозиторію на платформі GitHub (Рис. 6).

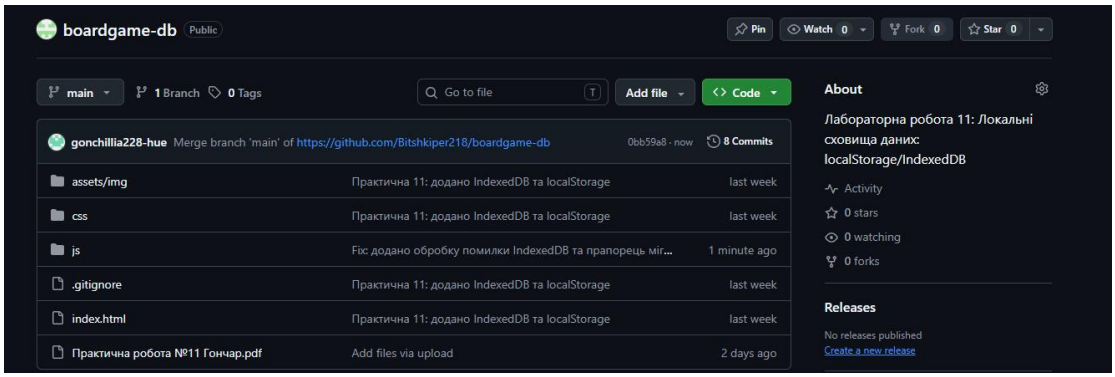


Рис. 6 — Фіксація змін у системі контролю версій Git на платформі GitHub

Посилання на репозиторій:

https://github.com/Bitshkiper218/boardgame-db

Висновок:

Під час виконання практичної роботи було успішно освоєно принципи роботи з локальними сховищами даних у браузері. На першому етапі було реалізовано синхронне збереження стану застосунку за допомогою localStorage та фреймворку Vue (використано метод watch із параметром deep: true). На другому етапі застосунок було переведено на використання повноцінної асинхронної бази даних IndexedDB із реалізацією CRUD-операцій.

Для збереження накопиченого користувацького досвіду успішно налаштовано механізм одноразової міграції даних із localStorage до IndexedDB із встановленням прапорця підтвердження. Додано коректну обробку помилок доступу до бази даних із виведенням повідомлень як у консоль, так і для користувача (через alert). Коректність роботи обох сховищ перевірено шляхом тестування інтерфейсу після перезавантаження сторінки (F5) та в панелі DevTools. Фінальний код проєкту успішно версіоновано та збережено у віддаленому репозиторію на платформі GitHub.